

Stoquify Agent and Skill Leverage Report

Date: 2026-07-21

Scope: Repository-grounded and internet/GitHub-grounded recommendation for agents and skills that can raise Stoquify's professional level with the least code and the highest leverage.

Executive Decision

Stoquify should not become a generic autonomous-agent platform. It should become an evidence-backed operating copilot for stock, cash, controls, close, compliance, payroll, and owner action. The best leverage is a thin agent control plane around the services and proof trails Stoquify already owns.

Highest-return path:

1. Build one shared agent runtime inside Stoquify, not one framework per module.
2. Use TypeScript-native building blocks first: Vercel AI SDK or similar TS-native tool loops, Model Context Protocol tools, Inngest/Trigger.dev-style durable execution, and Langfuse/Helicone/LiteLLM-style observability and model routing.
3. Keep domain skills Stoquify-native because the moat is tenant-scoped evidence, RBAC, OHADA context, stock-to-cash-to-close continuity, and controlled workflow execution.
4. Launch three user-facing agents first: Command Agent, Cash/Reconciliation Agent, Inventory/Replenishment Agent. Treat Exception and Action Orchestrator as platform infrastructure.
5. Do not give any agent direct Prisma write access, direct ledger posting, direct statutory filing, or permission-changing authority.

Local Stoquify Evidence

- `package.json` shows a Next.js, Prisma, Better Auth, TanStack Query, Jest, Playwright, ESLint, TypeScript, and Zod stack. The scan did not show a production agent framework dependency.
- `docs/architecture/decisions/0001-actions-vs-services.md` establishes the key rule: business logic belongs in `services/`, while server actions are thin wrappers.
- `prisma/schema.prisma` is broad and enterprise-shaped: tenant identity, inventory, purchasing/AP, POS, offline sync, payments, reconciliation, accounting, close assurance, compliance, HRIS, payroll, workflow assurance, audit, fiscal documents, and public boundary controls.
- `docs/agents` and `skills/STOQUIFY_ENTERPRISE_AGENT_AND_SKILL_SYSTEM_ASSESSMENT_2026-07-15.md` already concludes that Stoquify is agent-ready in data but not yet in runtime governance. It also records no production agent-run/tool-execution/prompt-version/model-provider/approval runtime in app code or schema.
- `docs/copilot/stoquify-agent-skill-definition-suite/registry/capability-registry.md` already defines 28 skills and 9 agents. This report extends that local work with broader GitHub/official-source research and a more selective build order.
- `what-next/ci-release-readiness.md`, `what-next/ledger-close-truth-readiness.md`, and `what-next/report-trust-export-readiness.md` show several gates ready.
- `what-next/release-secret-preflight.md` is conditional and `what-next/statutory-country-pack-production-readiness.md` is blocked; these are important no-autonomy boundaries.
- Graphify reports exist under `app/graphify-out`, `components/graphify-out`, `actions/graphify-out`, `services/graphify-out`, `hooks/graphify-out`, and `types/graphify-out`. They show route, service, component, and snapshot/action communities, but the canonical five filenames listed in `AGENTS.md` were not present at the root.

External GitHub and Official Source Scan

The recommendation is not to adopt all of these. Use them as design references and select a small production stack.

External project or standard	What it offers	Fit for Stoquify	Pros	Cons / complication	Verdict
OpenAI Agents SDK	Lightweight agents, tools, handoffs, guardrails, tracing. Sources: https://github.com/openai/openai-agents-python and https://openai.github.io/openai-agents-python/	Strong as conceptual/runtime reference.	Few primitives, guardrails, tracing, good mental model.	Python-first source; Stoquify is TS/Next.	Use patterns; adopt only through a TS-compatible route.
Vercel AI SDK	TypeScript SDK for AI apps, tools, approvals, workflow agents, skills, MCP apps. Source: https://github.com/vercel/ai	Very strong.	Fits Next.js, Zod, streaming UI, provider abstraction.	Requires careful approval and trace design.	Best low-code app-layer candidate.
Model Context Protocol TypeScript SDK	Standard way to expose tools/resources/prompts to agents. Source: https://github.com/modelcontextprotocol/typescript-sdk	Very strong.	Vendor-neutral tool boundary for Stoquify services.	v2 was pre-alpha in crawled source; stay on stable v1 for production.	Build Stoquify service tools behind MCP-compatible contracts.
Inngest AgentKit / Durable Agents	Durable agent loops, checkpointing, event waits, human-in-loop. Sources: https://github.com/inngest/agent-kit and https://www.inngest.com/docs/learn/durable-agents	Very strong.	Resumability, retries, approval waits, step traces.	Adds execution infrastructure.	High leverage for approval workflows; pilot narrowly.
Trigger.dev	TypeScript durable tasks, retries, queues, observability, human-in-loop. Source: https://github.com/triggerdotdev/trigger.dev	Strong alternative to Inngest.	Good Next/TS fit, self-host option.	Overlaps Inngest.	Choose either Inngest or Trigger.dev, not both.
Langfuse	Open-source LLM observability, evals, prompt management, datasets. Source: https://github.com/langfuse/langfuse	Very strong.	Self-hostable traces, evals, prompt versions.	Redaction and retention must be designed.	Best candidate for agent observability/evals.
Helicone	LLM observability and AI gateway with routing/fallbacks/cost tracking. Source: https://github.com/Helicone/helicone	Strong.	Fast integration, cost/latency, routing, prompts.	Gateway becomes sensitive-data chokepoint.	Good alternative/complement to Langfuse.
LiteLLM	AI gateway for many providers with budgets, keys, routing, guardrails. Source: https://github.com/BerriAI/litellm	Strong if multi-provider routing matters.	Spend control and fallback.	Python service overhead.	Worth if provider abstraction is strategic.
Pydantic AI	Type-safe Python agent framework with evals and observability. Source: https://github.com/pydantic/pydantic-ai	Moderate.	Excellent typed outputs and eval discipline.	Python-first.	Use for design inspiration unless Python workers are accepted.
LangGraph	Stateful, long-running agent orchestration. Source: https://github.com/langchain-ai/langgraph	Moderate to strong.	Mature graph/state patterns.	Heavy framework gravity.	Use only if TS-native options fall short.
Microsoft Agent Framework	Python/.NET production agent framework; successor direction for AutoGen/Semantic Kernel. Source: https://github.com/microsoft/agent-framework	Moderate.	Enterprise posture, workflows, MCP/A2A patterns.	Less direct for Next core.	Track if Azure/Microsoft ecosystem becomes core.
CrewAI	Role/task/crew abstractions for multi-agent collaboration. Source: https://github.com/crewAIInc/crewAI	Moderate as prototyping reference.	Easy specialist-role modeling.	Can encourage over-broad autonomous teams.	Use for ideation, not Stoquify runtime.
Mastra	TypeScript agents, tools, workflows, memory, observability. Source: https://github.com/mastra-ai/mastra	Strong alternative candidate.	TypeScript-native and workflow-aware.	Adds framework layer.	Evaluate against Vercel AI SDK; choose one.
Agno	Agent platform/control plane with RBAC, tracing, scheduling, integrations. Source: https://github.com/agn-agi/agno	Moderate.	Control-plane thinking matches Stoquify.	Python platform may be too large.	Borrow patterns; avoid full adoption initially.
Haystack	Production RAG and context-engineered pipelines. Source: https://github.com/deepset-ai/haystack	Moderate.	Transparent retrieval pipelines.	Python-centric and RAG-heavy.	Use if documentation/evidence retrieval expands.

External project or standard	What it offers	Fit for Stoquify	Pros	Cons / complication	Verdict
Dify	Visual workflow studio for agentic workflows. Source: https://github.com/langgenius/dify	Moderate as prototype/admin reference.	Human review, traces, visual flow.	Separate runtime and licensing/data-boundary review.	Prototype only, not embedded core.
Flowise	Visual AI agent builder and workflows. Source: https://github.com/FlowiseAI/Flowise	Low to moderate.	Fast demos.	Security and enterprise-boundary caution.	Avoid production core.
OpenHands	AI-driven development platform, CLI/SDK/sandboxed coding agents. Source: https://github.com/OpenHands/openhands	Strong for engineering, not product runtime.	Self-hosted coding automation.	Powerful tool execution needs guardrails.	Use for dev workflow experiments.
Cline	Open-source coding agent with SDK, IDE extension, CLI, rules, skills, MCP. Source: https://github.com/cline/cline	Strong for team coding workflow.	Human approvals, repo skills/rules, MCP.	Can make broad edits if uncontrolled.	Useful for dev productivity; not product runtime.
Continue	Source-controlled AI checks and IDE assistant. Source: https://github.com/continuedev/continue	Strong for code quality automation.	Markdown checks, CI-enforceable review gates.	Product direction should be monitored.	Use concept: repo-native AI checks.
Aider	Terminal pair programming with repo map, git integration, lint/test loop. Source: https://github.com/Aider-AI/aider	Moderate for local coding.	Low setup, focused edits.	Not enterprise product runtime.	Good individual dev tool.
SWE-agent / mini-SWE-agent	GitHub issue fixing and SWE-bench style automation. Source: https://github.com/SWE-agent/SWE-agent	Moderate for benchmarked repair.	Useful issue-to-patch/eval model.	Risky in dirty worktrees if unattended.	Use for evaluation inspiration.
Goose	General-purpose local AI agent with MCP/ACP, desktop/CLI/API. Source: https://github.com/aaif-goose/goose	Moderate for local operations.	Open-source, MCP, many providers.	Broad tool access increases risk.	Dev/operator experiments only.
Anthropic Agent Skills	Skill standard and public examples. Source: https://github.com/anthropics/skills	Very strong for skill structure.	Portable skill folders, scripts, resources, examples.	Vendor-specific limits differ.	Use as a template pattern.
GitHub Agentic Workflows	Markdown-defined repo automations in GitHub Actions. Source: https://docs.github.com/en/copilot/concepts/agents/about-github-agentic-workflows	Strong for repository maintenance.	Natural-language CI tasks with permissions/guardrails.	Requires careful repo permissions.	Good for CI failure summaries and docs drift.

Recommended Minimal Production Stack

1. Vercel AI SDK or equivalent TS-native agent loop for chat, tools, streaming, structured outputs, and approvals.
2. MCP TypeScript SDK for exposing Stoquify read models and controlled service commands as tools.
3. Inngest AgentKit or Trigger.dev for durable multi-step workflows that wait for approval and resume safely.
4. Langfuse or Helicone for traces, prompt versions, eval datasets, feedback, cost, and failure analysis.
5. LiteLLM only if multi-provider routing, budgets, and gateway-level guardrails become urgent.

Avoid introducing Python-first frameworks into the core Next.js product until there is a clear worker boundary.

20 Recommended Stoquify Agents

#	Agent	What it brings	Pros	Cons / complications	Worth using?
1	Stoquify Command Agent	Daily role-aware operating brief across stock, cash, close, payroll, and actions.	Highest daily habit potential; uses existing dashboards/action queues.	Needs strict evidence links and unavailable-answer behavior.	Yes, build first in read-only mode.
2	Exception and Action Orchestrator	Shared routing, approval, state, escalation, retries, and handoff core.	Prevents one-off agents per module; centralizes safety.	Requires new agent run/state/evidence records.	Yes, platform core, not a chat persona.

#	Agent	What it brings	Pros	Cons / complications	Worth using?
3	Cash and Reconciliation Agent	Explains cash mismatches, suspense, mobile-money/provider issues, and reconciliation blockers.	Direct financial value and strong evidence base.	Must never post, reverse, or certify autonomously.	Yes, first domain MVP.
4	Inventory and Replenishment Agent	Location-specific stock risk, replenishment drafts, variance investigation, supplier commitment awareness.	High-frequency operational value.	Offline/provisional stock and transfer state must be explicit.	Yes, first domain MVP.
5	Purchasing and AP Variance Agent	Links PO, receipt, invoice, supplier bank, payment risk, and three-way match exceptions.	Strong cash-control and fraud-prevention value.	High-risk actions require approval/fresh auth.	Yes, after cash/inventory.
6	Close and Compliance Agent	Navigates close blockers, evidence gaps, close packs, compliance readiness, and unsupported-country refusals.	Turns proof trails into accountant-ready workflows.	Statutory automation must fail closed.	Yes, after foundation.
7	Platform Assurance Agent	Watches agent runs, policy violations, eval drift, cost, latency, and incident patterns.	Essential for enterprise trust.	Needs observability data and incident model.	Yes, build before broad rollout.
8	Payroll and Workforce Readiness Agent	Explains payroll input readiness, aggregate variance, country-pack provenance, and privacy-safe blockers.	Strong HR/payroll workflow fit.	Sensitive payroll data and country rules raise risk.	Yes, later and read-only first.
9	POS Cash Drawer Control Agent	Daily drawer anomalies, cash shortage policy, terminal issues, void/refund patterns.	Immediate SMB operating value.	Must not accuse users or auto-adjust cash.	Yes, after cash agent.
10	Offline POS Replay Agent	Explains sync gaps, replay certificates, conflicts, provisional sales, and fiscal replay readiness.	Differentiates Stoquify for low-connectivity markets.	Conflict resolution is high risk.	Yes, as specialist under POS/cash.
11	Module Control Plane Agent	Surfaces module ownership, entitlement gaps, leakage risks, and rollout certification state.	Raises SaaS packaging maturity.	Must not change entitlements autonomously.	Yes, internal/admin first.
12	Security/RBAC Boundary Agent	Reviews route/action/service permissions, fresh-auth needs, redaction, and tenant leakage.	Reduces security drift with little product UI code.	False positives can slow delivery.	Yes, CI/report-first.
13	Report Trust Export Agent	Checks exports for provenance, currency, period status, redaction, hash, and certification boundary.	Supports accountant trust and audit readiness.	Export semantics must stay service-owned.	Yes, high leverage.
14	Country Pack Provenance Agent	Explains which country rules are supported, reviewed, sandbox-only, or blocked.	Critical for OHADA credibility.	Legal/regulatory claims must be constrained.	Yes, but no production filing autonomy.
15	Release and CI Gate Agent	Summarizes verify failures, release blockers, secret preflight, policy gates, and test impact.	Strong engineering leverage with minimal app code.	Must not hide unrelated failures.	Yes, developer/internal first.
16	Evidence Graph Root-Cause Agent	Follows proof links across sale, payment, stock, ledger, close, payroll, and compliance events.	Stoquify moat: cross-domain continuity.	Needs trustworthy evidence links and freshness.	Yes, as read-only reasoning layer.
17	Model Cost and Routing Agent	Routes tasks by risk, budget, latency, and model capability.	Controls spend and performance.	Wrong routing can degrade quality.	Yes, after telemetry exists.
18	Onboarding and Data Migration Agent	Guides setup, imports, mapping gaps, pilot migration, and readiness signoff.	Helps activation without huge UI rebuild.	Bad imports can poison data.	Yes, controlled and reversible.
19	Customer Success Adoption Agent	Explains value moments, next best action, training gaps, and blocked workflows per tenant.	Turns Stoquify into a habit and retention engine.	Needs product analytics not yet proven.	Later; build after telemetry.
20	Developer Quality and PR Agent	Uses repo rules, graph reports, tests, and policy gates to review changes and draft patches.	High engineering leverage; can use external coding agents safely.	Dirty worktrees and broad edits are risky.	Yes, internal with strict scope.

20 Recommended Stoquify Skills

#	Skill	What it brings	Pros	Cons / complications	Worth using?
1	Trusted Context Resolver	Resolves tenant, actor, role, route, module, period, location, and feature flags before any answer/tool.	Prevents context drift and tenant confusion.	Must be server-owned and cache-safe.	Yes, mandatory foundation.
2	Permission and Entitlement Guard	Checks RBAC, module entitlement, fresh-auth, SoD, route/action risk, and denial reason.	Turns agents into policy consumers, not policy bypassers.	Requires complete permission mapping.	Yes, mandatory foundation.
3	Evidence-Grounded Retrieval	Retrieves only approved service/read-model evidence with source, freshness, grade, and redaction.	Reduces hallucinated financial claims.	Requires evidence metadata discipline.	Yes, mandatory foundation.
4	Safe Action Planner	Produces draft plans, preconditions, blast radius, approvals, and rollback/no-op states.	Useful before controlled writes.	Can feel slow if every action is over-planned.	Yes, for high-risk workflows.
5	Approval Step-Up Coordinator	Routes fresh auth, manager/controller approval, accountant signoff, and denial outcomes.	Keeps autonomy bounded.	Adds workflow friction.	Yes, for finance/payroll/compliance.
6	Idempotent Tool Executor	Invokes allowlisted deterministic service commands with schemas, idempotency keys, timeouts, and typed errors.	Safest path to controlled writes.	Needs tool registry and error taxonomy.	Yes, before any write-capable agent.
7	Agent Evidence Recorder	Persists prompt/model/tool/skill versions, evidence fingerprints, approvals, cost, and terminal state.	Makes runs auditable and debuggable.	Storage/privacy design required.	Yes, mandatory.
8	Redaction and Disclosure Policy	Applies data classification, minimization, redaction, and unsupported-disclosure refusals.	Critical for payroll, identity, banking, public receipt, and accountant exports.	Requires careful tests.	Yes, mandatory.
9	Freshness Trust Evaluator	Labels answers as live, stale, partial, blocked, or unsupported.	Makes recommendations safer.	Needs observed-at and source freshness metadata.	Yes, mandatory.
10	Exception Prioritizer	Ranks issues by cash impact, compliance risk, stockout risk, close blocker severity, and SLA.	Converts noise into action.	Scoring can be political if opaque.	Yes, with explainable scoring.
11	Notification Escalation Router	Chooses in-app, email, digest, manager escalation, or silence based on risk and preferences.	Prevents alert fatigue.	Requires preference and quiet-hour rules.	Yes, after prioritizer.
12	Agent Run State Machine	Handles queued, running, waiting approval, blocked, retryable, denied, failed, completed, cancelled, degraded.	Enables durable workflows and safe resume.	Needs schema and operational UI.	Yes, mandatory.
13	Model Cost Router	Chooses model/provider by task risk, budget, language, latency, and evidence load.	Controls costs and quality.	Bad routing can reduce trust.	Yes, after initial telemetry.
14	Offline Replay Awareness	Distinguishes online, offline, provisional, replayed, conflict, certified, and fiscal replay states.	Unique value for African SMB contexts.	Complex edge cases.	Yes, POS/inventory/cash priority.
15	Country Pack Provenance Resolver	Explains country-pack source, review status, effective dates, sandbox/production boundary, and unsupported automation.	Protects compliance credibility.	Legal boundary must be explicit.	Yes, read-only first.
16	Daily Operating Brief	Produces owner/manager/cashier/accountant daily view with action queue and proof links.	Highest adoption leverage.	Requires concise UX and evidence links.	Yes, first user-facing skill.
17	Cross-Domain Root-Cause Trace	Traces sale/payment/stock/supplier/payroll/ledger/close dependencies.	Differentiated moat.	Needs source-link quality.	Yes, read-only MVP.
18	Cash Exception Triage	Groups cash mismatches, provider delays, drawer issues, suspense, and reconciliation blockers.	Immediate financial control value.	Must avoid unsupported fraud claims.	Yes, first domain skill.
19	Reconciliation Match Suggestion	Suggests matches with confidence, evidence, alternatives, and human approval.	Reduces reconciliation time.	False positives can damage books.	Yes, draft-only initially.
20	Inventory Risk and Replenishment	Forecasts stockouts, overstock, supplier delays, reorder drafts, and variance causes.	High operational ROI.	Must account for offline/provisional stock.	Yes, first domain skill.

Adoption Order

Days 0-30: Runtime foundation

- Add minimal agent tables: `AgentDefinition`, `AgentSkillVersion`, `AgentRun`, `AgentStep`, `AgentEvidenceLink`, `AgentApproval`, `AgentFeedback`, `AgentCost`, `AgentIncident`.
- Add a versioned tool registry that points to existing `services/` methods or read models.
- Add Trusted Context Resolver, Permission/Entitlement Guard, Evidence Retrieval, Redaction, Freshness, Evidence Recorder, Run State Machine.
- Add Langfuse or Helicone-style trace/eval/cost tracking.
- Ship a read-only Command Agent to internal users only.

Days 31-60: First business pilot

- Launch Cash/Reconciliation Agent and Inventory/Replenishment Agent in read-and-draft mode.
- Add human approval workflow for draft-only recommendations.
- Add feedback labels: accepted, rejected, stale, wrong, incomplete, unsafe, duplicate.
- Track activation, recommendation acceptance, time-to-resolution, correction rate, and unavailable-answer rate.

Days 61-90: Controlled write boundary

- Allow only low-risk service commands through Idempotent Tool Executor.
- Keep ledger posting, statutory filing, permission changes, payroll mutation, cash adjustment, stock write-off, and close certification human-owned.
- Add Platform Assurance Agent to watch policy violations, latency, cost, and drift.

High-Leverage Engineering Work

1. Do not rewrite services. Wrap existing services and read models as tools.
2. Do not add a generic memory system first. Use explicit, per-user preferences only after redaction and retention policy are in place.
3. Do not install a full visual agent studio into the product core. Use visual tools only for internal prototyping.
4. Create a small `lib/agents/` or `services/agent-control/` boundary that owns run state, skills, tool registry, and evidence links.
5. Use JSON schemas/Zod for every skill input and output.
6. Add tests that assert denied tools cannot be retried through a weaker tool.
7. Treat graphify reports as advisory evidence, not canonical truth, until refreshed and normalized.

What Not To Do

- Do not build 20 autonomous agents at once.
- Do not let agents write directly to Prisma.
- Do not allow autonomous ledger posting, statutory filing, payroll mutation, entitlement changes, or role changes.
- Do not copy sensitive raw records into agent memory.
- Do not create one agent per route or one agent per module.
- Do not adopt multiple agent frameworks at once.
- Do not treat external coding agents as production product agents.

Final Recommendation

Build Stoquify's agent system as a controlled, evidence-linked operating copilot. The top priority is not "more AI"; it is agent governance, source-linked answers, durable approval workflows, cost/trace observability, and three high-frequency agents: Command, Cash/Reconciliation, and Inventory/Replenishment.

The least-code/highest-leverage path is TS-native orchestration plus Stoquify-native skills over existing services. External projects should supply runtime patterns, tool protocols, durability, and observability; Stoquify should own the domain skills, evidence contracts, and approval boundaries.